

Module – 3

Indexing and Searching in Web Information Retrieval

Topics:

1. Indexing and Searching
2. Inverted indexes
3. Sequential searching
4. The web
5. Search engine Architectures
6. Cluster based architecture
7. SE ranking
8. Link based ranking
9. Simple ranking functions
10. Evaluations
11. Application of web crawlers

Indexing and Searching

Information Retrieval (IR) aims to find relevant information from large collections of documents in response to user queries.

Two critical processes make this possible:

1. **Indexing** – Organizing and structuring data for fast access.
2. **Searching** – Retrieving documents that best match the user's query using the index.

Need for Indexing

When millions of documents are stored (e.g., on the web), **scanning each document** to find a query term is extremely slow.

We need a **data structure** that quickly points to documents containing a given term.

Example:

Suppose we have 3 documents:

- D1: "AI improves healthcare"
- D2: "Healthcare uses data analytics"
- D3: "AI and data drive automation"

Without indexing → the system reads every document linearly (slow).

With indexing → we instantly know where "AI" appears: {D1, D3}.

What is an Index?

Definition:

An **index** is a data structure that stores **terms** and their corresponding **document references**, allowing efficient retrieval.

Analogy:

Just like the **index of a book** tells you on which pages a term occurs, an IR index tells you in which documents a word appears.

Types of Indexes

1. Forward Index

- Stores, for each document, the list of all terms it contains.
- Useful for building inverted indexes.
- Example:
D1 → {AI, improves, healthcare}
D2 → {Healthcare, uses, data, analytics}

2. Inverted Index

- The most common structure in IR systems.
- For each **term**, it lists all documents that contain it.
- Enables fast search and Boolean queries.

Term	Documents
AI	D1, D3
healthcare	D1, D2
data	D2, D3
analytics	D2
automation	D3

Search “AI AND data” → returns D3

3. Positional Index

A **Positional Index** is an **extended version of the inverted index** that not only stores *which documents* contain a term, but also the **positions (word offsets)** of that term **within each document**.

Term	Posting List
machine	(D1: 2, 10), (D2: 5), (D3: 1, 7)
learning	(D1: 3, 11), (D2: 6), (D3: 2, 8)

Interpretation:

- “machine” occurs in D1 at positions 2 and 10, in D2 at position 5, and in D3 at positions 1 and 7.
- “learning” occurs in D1 at positions 3 and 11, etc.

Now we can identify **phrases** like “machine learning” — where “learning” follows “machine” immediately.

Example 1 – Phrase Query

Documents

Doc ID	Text
D1	“machine learning improves healthcare”
D2	“deep learning uses neural networks”
D3	“machine intelligence and machine learning”

Step 1: Build Positional Index

Term	Posting List
machine	D1:1, D3:1,4
learning	D1:2, D2:2, D3:5
improves	D1:3
healthcare	D1:4
deep	D2:1
uses	D2:3
neural	D2:4
networks	D2:5
intelligence	D3:2
and	D3:3

Step 2: Phrase Query = “machine learning”

We need documents where “**learning**” follows “**machine**” immediately (position of learning = position of machine + 1).

Checking each document:

- D1 → machine(1), learning(2) ✓ **match**
- D2 → no “machine” ✗
- D3 → machine(4), learning(5) ✓ **match**

Result: D1, D3

Example 2 – Proximity Query (NEAR operator)

Query:

Find documents where “**machine**” and “**learning**” occur **within 2 words of each other**.

Use the same positional index as above.

Check:

Doc	machine positions	learning positions	Distance
D1	1	2	1 ✓
D3	4	5	1 ✓

Both D1 and D3 satisfy the “within 2 words” condition.

Searching

Once the index is built, searching becomes fast and efficient.

Basic Search Process

1. User enters a query → e.g., “AI healthcare”
2. System parses query → removes stop words, stems terms.
3. Index lookup → find documents containing “AI” and “healthcare”.
4. Rank documents → based on retrieval model (e.g., Boolean, Vector Space, Probabilistic).
5. Display results → sorted by relevance.

Search Techniques

(a) Boolean Searching

- Uses logical operators (AND, OR, NOT).
- Example:
Query: (AI AND healthcare) → Returns D1.
Query: (AI OR data) → Returns D1, D2, D3.

(b) Vector Space Searching

- Documents and queries are represented as vectors.
- Similarity measured by cosine similarity.
- Example: Query = “data analytics”
 - D2 is most similar (since both terms occur there).

(c) Probabilistic Searching

- Ranks documents based on probability of relevance.
- Uses models like **Binary Independence Model (BIM)**.

(d) Fuzzy Searching

- Handles typos and approximate matches.
- Example: Searching “analytics” may still match “analytcs”.

Benefits of Indexing and Searching

- Fast access to information.
- Efficient storage and retrieval.
- Enables scalable search engines.

Real-Life Examples

Platform	Indexing Used	Search Behavior
Google	Inverted index + link analysis	Returns ranked web pages
PubMed	Keyword-based and MeSH indexing	Finds biomedical research papers
YouTube	Metadata + content indexing	Returns videos
E-commerce (Amazon)	Product catalog index	Filters and ranks items
Library IR System	Author/title indexing	Searches books and journals

Challenges in Indexing and Searching

1. **Scalability:** Billions of documents must be handled efficiently.
2. **Dynamic updates:** Documents are added, modified, or deleted constantly.
3. **Synonyms and ambiguity:** Same word, different meanings.
4. **Multilingual content:** Requires language-specific pre-processing.
5. **Real-time search:** Instant results for web-scale systems.

Steps in Building an Inverted Index

1. Document Parsing

Definition

Document parsing is the **first step** in index construction. It means **reading and processing each document** in the collection to extract meaningful text.

What Happens

- Convert documents into a **uniform format** (plain text).
- Remove **metadata**, HTML tags, or formatting symbols.
- Identify **fields** (title, author, body, etc.) if structured.

Example

If a webpage contains:

```
<title>AI in Healthcare</title>  
<p>AI improves healthcare outcomes.</p>
```

After parsing:

AI in Healthcare AI improves healthcare outcomes

Now it's ready for tokenization.

Why It's Important

- Documents come in different formats (PDF, Word, HTML, JSON).
- Parsing extracts only the **indexable content**.
- Prevents noise (like tags, special characters) from entering the index.

2. Tokenization

Definition

Tokenization is the process of **splitting text into individual words (tokens)**.

Process

1. Identify word boundaries (spaces, punctuation).
2. Convert all tokens to **lowercase**.
3. Remove special symbols or numbers if unnecessary.

Example

Input:

```
"AI improves healthcare"
```

After Tokenization:

```
[ai, improves, healthcare]
```

Edge Cases

Input	Tokens
“COVID-19 pandemic”	[covid, 19, pandemic]
“Dr. John’s research”	[dr, john, research]

Why It’s Important

- The inverted index stores **terms**, not full sentences.
- Proper tokenization ensures **consistent indexing** and better search matching.

3. Stop Word Removal

Definition

Stop words are **common, high-frequency words** that carry little or no meaning in retrieval (e.g., “the”, “is”, “of”, “and”, “to”).

Process

Remove them from the token list to **reduce index size** and **improve efficiency**.

Example

Before:

```
["ai", "is", "improving", "the", "healthcare", "system"]
```

After:

```
["ai", "improving", "healthcare", "system"]
```

Typical Stop Word List

{the, a, an, and, or, in, on, of, for, with, to, by, at, is, was, be, has, have}

Why It’s Important

- Saves storage space.
- Focuses on meaningful content words.
- Improves query precision.

(Note: Some modern systems keep stop words to support phrase queries like “to be or not to be.”)

4. Stemming / Lemmatization

Definition

These are techniques for **reducing words to their base or root form** so that similar words are treated as the same term.

Technique	Description	Example
Stemming	Cuts off prefixes/suffixes using simple rules.	“running”, “runs” → “run”
Lemmatization	Uses vocabulary and grammar rules to return base word (lemma).	“better” → “good”

Example

Before:

```
["computes", "computing", "computed"]
```

After:

```
["compute", "compute", "compute"]
```

Why It's Important

- Groups word variants under one index entry.
- Improves **recall** (more relevant results retrieved).
- Reduces vocabulary size and redundancy.

5. Term–Document Mapping

Definition

Now that we have clean tokens, the system creates a **mapping between terms and the documents** in which they appear.

Process

For each term:

- Add it to a **dictionary** (vocabulary) if new.
- Add the **document ID (docID)** to that term's **posting list**.

Example

Documents:

DocID	Tokens
D1	ai, improves, healthcare
D2	ai, in, robotics

Term–Document Mapping:

Term	Posting List
ai	D1, D2
improves	D1
healthcare	D1
in	D2
robotics	D2

Now this table forms the **core inverted index**.

Why It's Important

- Provides a fast lookup from term → document.
- Foundation for Boolean, Vector, and Probabilistic models.
- Enables logical operations like AND/OR/NOT.

6. Index Storage / Compression

Definition

Once the index is built, it's stored efficiently for **fast access and minimal space**.

Common Techniques

a) Posting List Compression

Instead of storing every document ID directly, store the **difference (gap)** between consecutive IDs.

Postings	Gap Encoding
[2, 5, 8, 10]	[2, 3, 3, 2]

Gaps are smaller → better compression.

b) Variable Byte Encoding

Store small numbers in fewer bytes using variable-length binary codes.

Example:

2 → 00000010 (1 byte)
300 → 100101100 (2 bytes)

c) Dictionary Compression

Since many terms share prefixes (e.g., “comput”, “computer”, “computing”),
→ Use **Front Coding**: store shared prefix once.

Terms	Stored As
compute	compute
computer	(r)
computing	(ing)

Why It's Important

- Large-scale IR systems deal with **billions of terms**.
- Compression reduces:
 - **Disk usage**
 - **I/O time**
 - **Query latency**

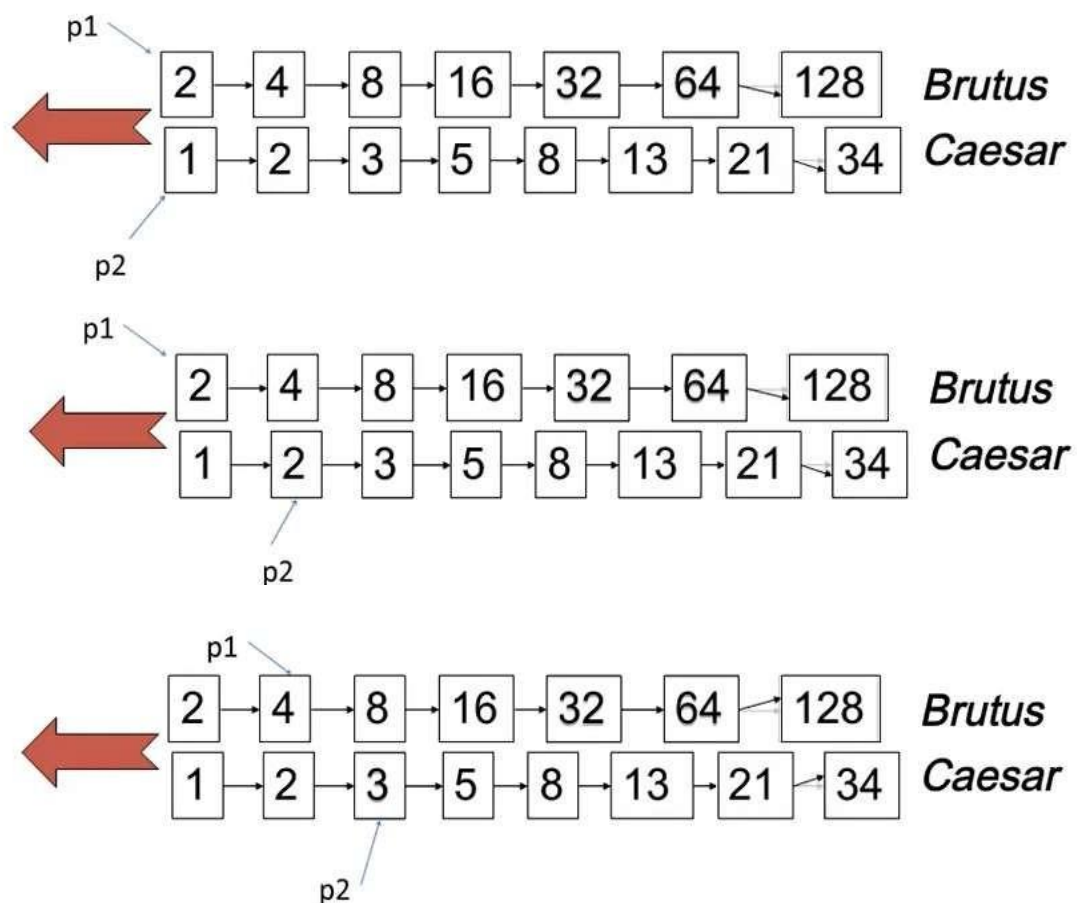
Intersecting two posting lists

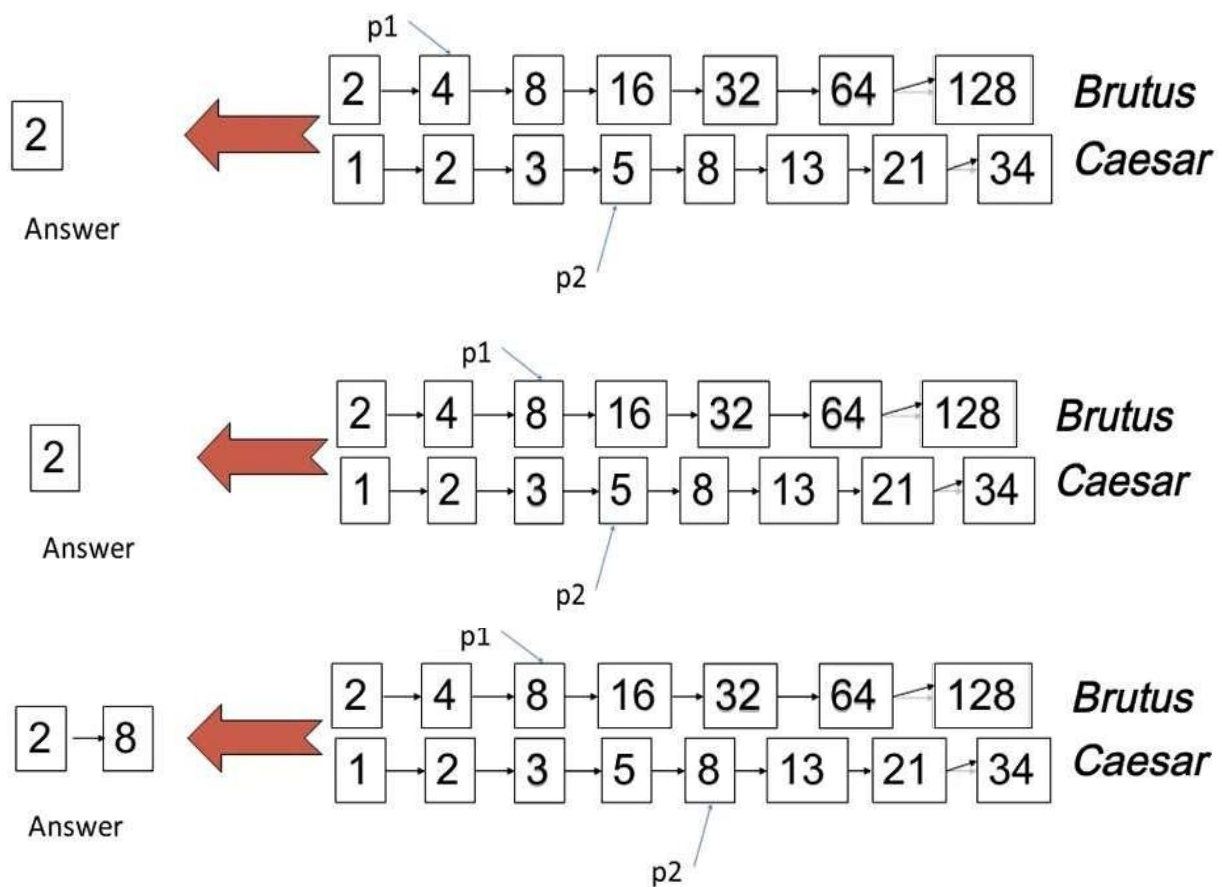
INTERSECT(p_1, p_2)

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(\text{answer}, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return answer
```

P1: pointer to current location in list1

P2: pointer to current location in list2





Sequential Searching

- Brute Force
- Knuth-Morris-Pratt

Brute – force algorithm

- Also known as Substring Search
- Given a text T and a pattern P,
 - Is the pattern a substring of the text, i.e.,
 - Is there a position I where the entire pattern occurs in the given text
 - Ignore partial matches

```

□ StringMatch(  $T[0..n-1]$ ,  $P[0..m-1]$ )
□ for  $i \leftarrow 0$  to  $n - m$  do
  □  $j \leftarrow 0$ 
  □ while  $j < m$  and  $P[j] = T[i + j]$  do
    ■  $j \leftarrow j + 1$ 
  □ if  $j = m$  return  $i$ 
□ return -1

```

Knuth – Morris – Pratt (KMP) Algorithm

Steps of KMP Algorithm

1. Preprocessing (LPS Array Computation)

- The algorithm first preprocesses the pattern to create the **Longest Prefix Suffix (LPS)** array.
- This array helps in skipping unnecessary character comparisons in the main search.

2. Pattern Searching

- The pattern is compared with the text.
- If a mismatch occurs, the LPS array helps in determining the next position to check without rechecking already matched characters.

We have to draw the pi table Pattern 1

1	2	3	4	5	6	7	8	9	10
A	B	C	D	A	B	E	A	B	F
0	0	0	0	1	2	0	1	2	0

A	B	C	D	E	A	B	F	A	B	C
0	0	0	0	0	1	2	0	1	2	3

A	A	B	C	A	D	A	A	B	C
0	1	0	0	1	0	1	2	3	4

A	A	A	A	B	A	A	C	D
0	1	2	3	0	1	2	0	0

Problem:

A	B	A	B	C	A	B	C	A	B	A	B	A	B	D
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Start j = 0;

String[i] == pattern[j+1]

<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>
<u>A</u>	<u>B</u>	<u>A</u>	<u>B</u>	<u>D</u>
<u>0</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>0</u>

Examples:

Text: "ABABDABACDABABCABAB"

Pattern: "ABABCABAB"

Text (T): "AAACAAAAAC"

Pattern (P): "AAAA"

Text (T): "ABCABCABDABCABCABCABC"

Pattern (P): "ABCABCABC"

Web Search

- **Web Search** refers to the process of retrieving relevant information from the **World Wide Web (WWW)** based on user queries.
A **Web Search Engine** is an IR system that crawls, indexes, and retrieves web pages to satisfy user information needs efficiently.
- **Examples:** Google, Bing, Yahoo

Characteristics of Web Search

Unlike traditional document retrieval (like in libraries or academic databases), **web search** deals with:

- **Huge scale** – billions of documents.
- **Dynamic content** – web pages are created, modified, and deleted constantly.
- **Heterogeneous data** – text, images, audio, video, multimedia.
- **Noisy data** – duplicate, spam, irrelevant, and unstructured content.
- **Hyperlinks** – connecting pages (used to estimate page importance).

Components of a Web Search Engine

- Web Crawler (Spider or Bot)
- Indexer
- Query Processor
- Ranking Module
- User Interface

Web IR: Differences from Traditional IR

Feature	Traditional IR	Web IR
Document Collection	Small, static, and controlled (e.g., academic papers, library databases)	Massive, dynamic, unstructured, and uncontrolled (billions of web pages)
Data Format	Text-based and structured	Unstructured and multimedia (text, images, videos, links)
Users	Specific and trained (researchers, librarians)	General public (diverse knowledge levels)
Queries	Short, specific, often keyword or Boolean-based	Very short, vague, or natural language queries
Indexing	Simple term-based indexing	Large-scale distributed indexing, including link and metadata indexing
Ranking	Based on textual relevance (TF-IDF, cosine similarity)	Combines textual, link-based (PageRank), and behavioral signals
Evaluation	Metrics like precision, recall, F1-score	Also considers user satisfaction, click-through rates, dwell time
Data Source Control	Controlled by an organization	Open and uncontrolled (anyone can publish)
Updates	Rare, static document sets	Frequent updates, additions, deletions
Spam/Noise	Minimal	High – includes spam, duplicates, low-quality content

Duplicate documents

- In Web Information Retrieval (Web IR), one of the major challenges faced by search engines is the presence of duplicate and near-duplicate documents.
- Because the web is *open*, *unstructured*, and *uncontrolled*, many websites contain the same or slightly modified copies of content — leading to redundancy, wasted storage, and poor search quality.

A **duplicate document** is a web page that has **identical or almost identical content** to another page.

Two documents are considered *duplicates* if the textual content, structure, and meaning are the same or very similar, even if their URLs or HTML layouts differ.

Example:

- <https://example.com/about-us>
- <https://example.com/about-us.html>

Both may have **exactly the same text**, though the URLs differ.

Type 1: Exact Duplicate Documents

- Two documents are exact duplicates if they have the same content in terms of characters, words, and structure — even if URLs differ.

Example:

- <https://news.com/article?id=101>
- <https://news.com/article?id=101&session=555>

Type 2: Near-Duplicate Documents

Two documents that are not identical, but very similar in terms of text content (e.g., 80–95% similar).

Example:

- Article A: “India wins cricket match by 6 wickets.”
- Article B: “India wins by six wickets in today’s cricket match.”
- Although not identical, they express the same information.

Web Crawling

A web crawler is an automated program that systematically browses the World Wide Web to collect and store web pages for indexing by a search engine.

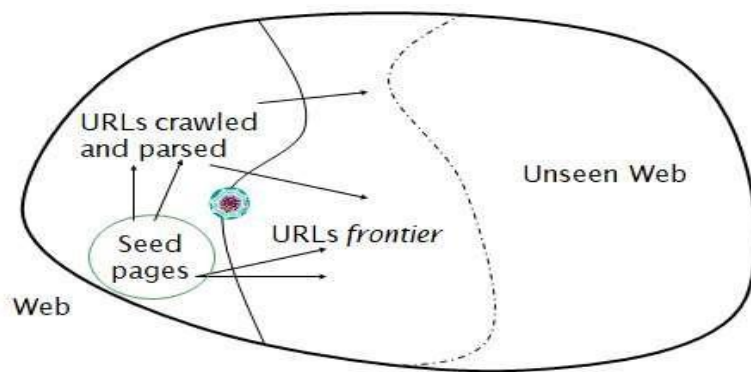
Example:

- Google → *Googlebot*
- Bing → *Bingbot*
- Yahoo → *Slurp*

Purpose of Web Crawling

Goal	Description
1. Content Discovery	Find and download new or updated web pages.
2. Data Collection	Gather web data for indexing, analytics, or research.
3. Freshness Maintenance	Ensure indexed data reflects the current version of web pages.
4. Link Structure Mapping	Understand how pages are connected (important for PageRank).

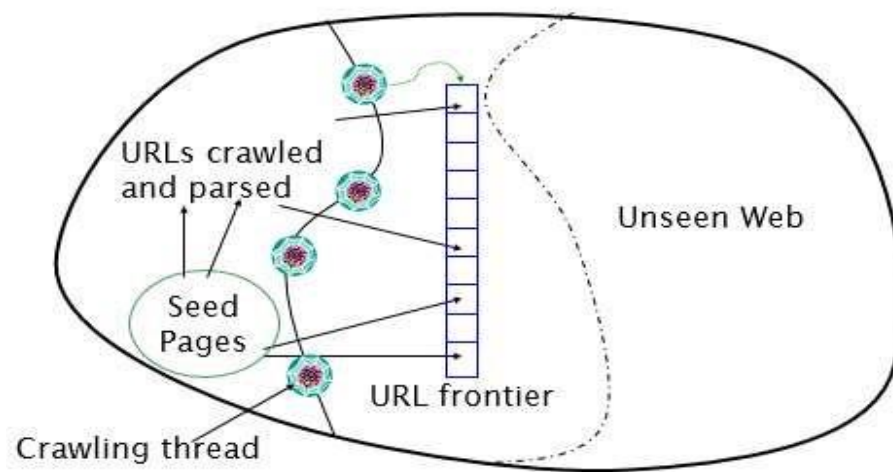
Crawling Picture



Features a Web Crawler must provide

- **Robustness:**
 - Be immune to spider traps, duplicates, very large pages, very large websites, dynamic pages etc.
 - **Politeness:**
 - **Explicit politeness:** specifications from webmasters on what portions of site can be crawled : **robots.txt**
 - **Implicit politeness:** even with no specification, avoid hitting any site too often
-
- **Distribution:** crawling should be distributed within several machines
 - **Scalability:** crawling should be extensible by adding machines, extending bandwidth, etc.
 - **Efficiency:** clever use of the processor, memory, bandwidth (e.g. as few idle processes as possible)
 - **Quality:** should detect the most useful pages, to be indexed first
 - **Freshness:** should continuously crawl the web (visiting frequency of a page should be close to its modification frequency)
 - **Extensibility:** should support new data formats (e.g. XML-based formats), new protocols (e.g. ftp), etc.
-

Updated Crawling Picture



URL Frontier

- The URL frontier is the data structure that holds and manages URLs we've seen, but that have not been crawled yet
 - Can include multiple pages from the same host
 - Must avoid trying to fetch them all at the same time
 - Must keep all crawling threads busy.
-

Robots.txt

- Protocol for giving spiders ("robots") limited access to a website, originally from 1994
 - www.robotstxt.org/wc/norobots.html
 - Website announces its request on what can(not) be crawled
 - For a server, create a file `/robots.txt`
 - This file specifies access restrictions.
-

- No robot should visit any URL starting with `"/yoursite/temp/"`, except the robot called "searchengine":

```
User-agent: *  
Disallow: /yoursite/temp/
```

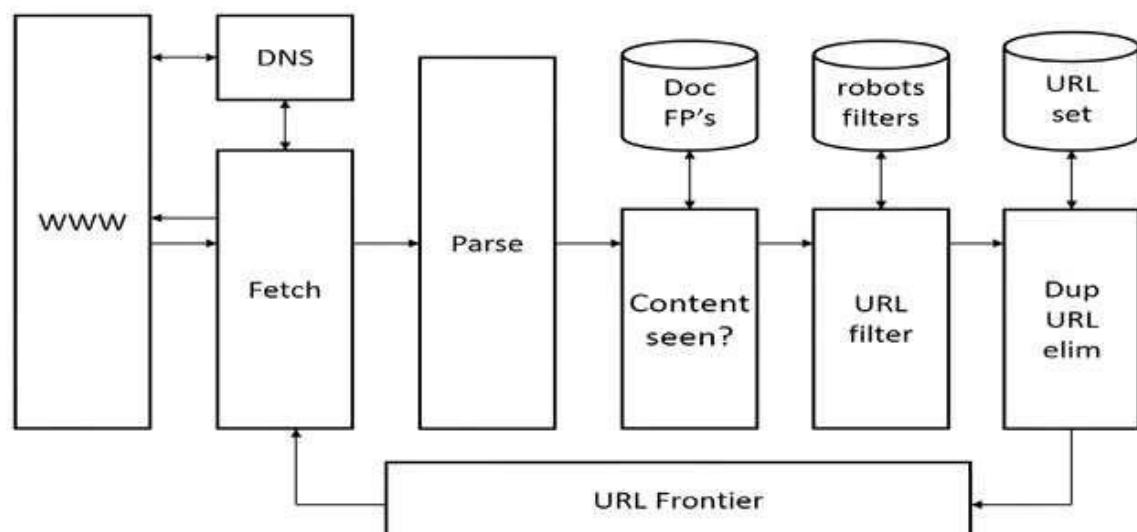
```
User-agent: searchengine  
Disallow:
```

Crawling process

- Pick a URL from the frontier
 - Fetch the document at the URL
 - Parse the URL
 - Extract links from it to other docs (URLs)
 - Check if URL has content already seen
 - If not, add to indexes
 - For each extracted URL
 - Ensure it passes certain URL filter tests
 - Check if it is already in the frontier (duplicate URL elimination).
-

Architecture of a Crawler

- **URL frontier** – managing the URLs to be fetched
 - **DNS resolution** – determining the host (web server) from which to fetch a page defined by a URL
 - **Fetching module** – downloading a remote webpage for processing
 - **Parsing module** – extracting text and links
 - **Duplicate elimination** – detecting URLs and contents that have been processed a short time ago
-



Crawler process in detail

- A crawler thread begins by taking a URL from the frontier and fetching the web page at that URL, generally using the HTTP protocol
 - The fetched page is parsed and the text as well as the links in it are extracted
 - The text (with HTML tags) is passed on to the indexer
 - Link information including anchor text is passed on to the indexer for use in ranking.
-

- Each extracted link goes through a series of tests to determine whether the link should be added to the URL frontier.
 - Content seen
 - URL filter
 - Duplicate elimination

Content seen

- Duplication is widespread on the web
 - If the page just fetched is already in the index, do not further process it
 - This is verified using document fingerprints or shingles.
-

Filters and robots.txt

- Filters – regular expressions for URL's to be crawled/not
 - Once a robots.txt file is fetched from a site, need not fetch it repeatedly
 - Doing so burns bandwidth, hits web server
 - Cache robots.txt files.
-

Duplicate URL elimination

- If the URL is already in the frontier or (in the case of a non-continuous crawl) already crawled, we do not add it to the frontier
 - When the URL is added to the frontier, it is assigned a priority based on which it is eventually removed from the frontier for fetching.
-

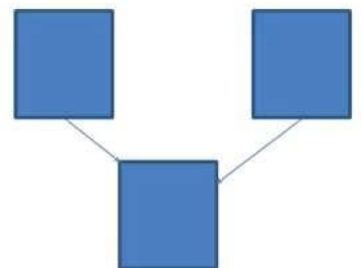
URL normalization

- When a fetched document is parsed, some of the extracted links are *relative* URLs
 - E.g., http://en.wikipedia.org/wiki/Main_Page has a relative link to /wiki/Wikipedia:General_disclaimer which is the same as the absolute URL http://en.wikipedia.org/wiki/Wikipedia:General_disclaimer
 - During parsing, must normalize (expand) such relative URLs
-

Introduction on PageRank

- PageRank is a link analysis algorithm ... with the purpose of "measuring" its (Webpage) relative importance within the set.
 - – From Wikipedia, the free encyclopedia
 - Developed by Larry Page as his PhD research topic
 - 3 years later, he quitted Stanford and founded Google with Brin
 - PageRank is a probability distribution used to represent the likelihood that a person randomly clicking on links will arrive at any particular page.
 - PageRank can be calculated for collections of documents of any size.
 - It is assumed in several research papers that the distribution is evenly divided among all documents in the collection at the beginning of the computational process.
-
- Page rank of a page P is the sum of links that point to P.
 - If there is one link pointing to P then the $PR(P) = 1$
 - If there is two links pointing to P then the $PR(P) = 2 \dots$
 - The probability of the user landing on page P based on the pages that point to P (step 1)

$$PR(P) = \sum \text{links points to P}$$

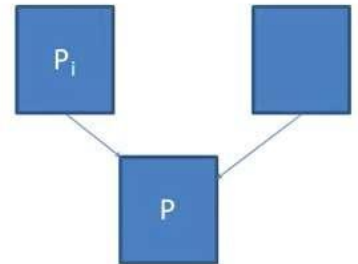


Step 2

- Assume a random walker and assume that there is a single path from Page A to P then he is more likely to reach P rather than any pages.
- The number of out going links from a page decreases if this number is high so divide this by the number of outgoing links

$$PR(P) = \sum \text{links points to } P / L(P_i)$$

Number of out going links from P_i



Step 3:

- Page P is more likely to be reached if there is a link from a page that has a big page rank(i.e there is a high probability of reaching this page)
 - Page rank of a page P is the sum of the page ranks of the page P_i that have a path to P divided by the out going links from P_i
 - $PR(P) = \sum PR(P_i) / L(P_i)$
-

Step 4: Random walker can move from one page to the other this model is captured by the Damping factor.

- Ability of the random walker to go anywhere is evaluated as per existing web documents and the value is 0.85 (i.e. after every six pages that will reach to a path).

N is total number of pages

$$= (1-d) / N + d \sum PR(P_i) / L(P_i)$$

i.e. Probability distribution

Two formula as per literature

$$PR(A) = (1-d) + d(PR(T_1)/L(T_1) + \dots + PR(T_n)/L(T_n))$$

$$PR(A) = (1-d)/N + d(PR(T_1)/L(T_1) + \dots + PR(T_n)/L(T_n))$$

Where $T_1, T_2 \dots T_n$ are pages that are pointing to A.

How is it Calculated ?

- The PR of each page depends on the PR of the pages pointing to it.
 - But we won't know what PR those pages have until the pages pointing to them have their PR calculated and so on.
 - So what we do is make a guess.
-

Examples:

Let's say there are 3 pages: A, B, and C.

Links:

- $A \rightarrow B$
- $B \rightarrow C$
- $C \rightarrow A$

Let's set:

- Initial PageRank of each page = 1
- Damping factor $d=0.85$

Pages: A, B, C, D

Link Structure:

- $A \rightarrow B, C$
- $B \rightarrow C$
- $C \rightarrow A$
- $D \rightarrow C$

Step 1: Setup

- $N=4, d=0.85$
- Initial $PR(A) = PR(B) = PR(C) = PR(D) = 1$

QUESTION BANK

2 MARKS QUESTION

1. Differentiate between indexing and searching
2. Explain the need for indexing
3. List some challenges in indexing and searching
4. List types of indexing.
5. Describe forward indexing with example
6. Describe inverted indexing with example
7. Describe positional indexing with an example.
8. List any 3 searching techniques.
9. Quote benefits of indexing
10. List 2 challenges of indexing and searching
11. List 2 important sequential searching methods
12. List characteristics of web search
13. Explain exact duplicate with example
14. Explain near duplicate with example

15. Define web crawlers
16. Describe page ranking.

10 MARKS QUESTION

1. Explain with examples types of indexing techniques
2. Explain types of indexing in detail.
3. List all steps in construction of inverted index. Explain any 2 in detail.
4. Solve for KMP sequential searching:

Examples:

Text: "ABABDABACDABABCABAB"

Pattern: "ABABCABAB"

Text (T): "AAACAAAAAC"

Pattern (P): "AAAA"

Text (T): "ABCABCABDABCABCABCABC"

Pattern (P): "ABCABCABC"

5. Difference between Traditional IR and web IR.
6. Explain the purpose of web crawlers
7. Explain in detail the features a web crawler must provide.
8. With architecture diagram, Explain the working of web crawlers.